

2PC Memory-Manipulating Programs with Constant Overhead

David Heath
University of Illinois Urbana-Champaign
daheath@illinois.edu

Abstract

General-purpose secure multiparty computation (MPC) remains bottlenecked in large part by a lack of efficient techniques for handling memory access. We demonstrate a remarkably simple and efficient 2PC instantiation of random access memory (RAM), based on distributed point functions (DPFs, Gilboa and Ishai, Eurocrypt’14). Our semi-honest 2PC protocol can be achieved from oblivious transfer (OT) and a black-box pseudorandom generator (PRG).

For a memory that stores large enough data words, our 2PC RAM incurs *constant* communication overhead per access. Like prior works that leverage DPFs to achieve memory access, our work incurs linear *computation* per access, but our per-access *communication* is lean.

Our 2PC RAM is built on top of an obliviousness-friendly model of computation called the single access machine model (SAM, Appan et al., CCS’24). In the SAM model, each memory slot can be read at most once. We present a simple 2PC SAM protocol, where each single-access memory operation incurs at most $2w + O(\lambda \lg n)$ bits of communication, where w is the word size, n is the number of memory words, and λ is a security parameter. Of this cost, only $2w + 2 \lg n$ bits are incurred in the online phase.

Our RAM operations are (non-cryptographically) compiled to SAM operations. At most a logarithmic number of SAM operations are needed per RAM operation; if word size is large, even fewer SAM operations are required. Alternatively, there are now many oblivious algorithms that compile directly to SAM more efficiently than via a compilation to RAM, and our 2PC SAM can instantiate these algorithms. As one example, we can use our 2PC SAM to implement privacy-preserving graph traversal (DFS or BFS) over a secret-shared size- n graph while revealing nothing beyond the runtime of the SAM program. Our construction achieves online communication $O(n \lg n)$ bits, asymptotically matching the number of bits touched in a corresponding cleartext graph traversal.

Keywords: MPC, Oblivious RAM, Oblivious Algorithms, Distributed Point Functions

Contents

1	Introduction	3
1.1	Contribution	4
1.2	Technical Overview	6
2	Preliminaries	7
2.1	Notation	7
2.2	Security Model and Functionalities	7
2.3	The Single Access Machine (SAM) Model	9
2.4	How to Implement RAM with SAM	10
2.5	Distributed Point Functions	11
3	Technical Approach	12
3.1	Protocol Details	12
3.2	Security Proof	14
4	Evaluation	15
4.1	Protocol Costs	17
4.2	Communication Cost Comparison	17
4.3	Efficiency of RAM to SAM Compilation	18
5	Related Work	18

1 Introduction

Secure multiparty computation (MPC) allows mutually untrusting parties to securely run programs on their joint, private inputs. This work focuses on two-party computation (2PC) protocols secure against a *semi-honest* adversary, and our goal is to improve *bandwidth consumption*. We in particular wish to improve 2PC cost for arbitrary and potentially complex programs.

We distinguish a protocol’s *offline* and *online* phases. The offline phase can be executed before party inputs are known, allowing parties to prepare “overnight”, while computational resources are not in contention. The goal in an offline-online protocol is to move cost to the offline phase, allowing for a lightweight online phase.

The problem of memory manipulation In theory, 2PC/MPC protocols allow parties to execute *arbitrary* programs $\mathcal{P}$ and the community’s foundational techniques work when $\mathcal{P}$ is expressed as a poly-size circuit [Yao86, GMW87]. Expressing $\mathcal{P}$ as a circuit can be problematic, particularly when $\mathcal{P}$ manipulates a large main memory. In general, compiling such a memory-manipulating program to a circuit can result in more than $\Omega(n)$ gates *per memory access*, due to the need to compile each access to a large subcircuit that linearly scans all of memory. While the resulting circuit remains poly-size, it is concretely large, and MPC’s communication cost typically scales with circuit size.

ORAM and RAM-MPC Fortunately, we can achieve significantly cheaper MPC for memory-manipulating programs by incorporating *Oblivious RAM* (ORAM). ORAM is a cryptographic technique allowing a memory-constrained client to securely outsource its data storage to a powerful, untrusted server [GO96]. In short, ORAM allows the client to obfuscate its logical access pattern, hiding from the server the logical address accessed at each step. Roughly, MPC and ORAM are combined by letting the MPC circuit play the role of the ORAM client, and letting the MPC parties jointly implement the ORAM server, where the parties hold *secret shares* of the ORAM server’s main memory; see [OS97, GKK⁺12, KM19, BKKO20, HV21, FNO22, VH23, FNO24] and others. This arrangement is sometimes called *distributed ORAM* (DORAM) or *RAM-MPC*.

Remark 1 (Multi-server ORAM versus RAM-MPC). *In some cases the term DORAM is used to mean a client/multi-server interaction, and in others it is meant to indicate a full MPC RAM, where the ORAM client is intended to be an MPC circuit. The former setting is easier than the latter, since in the former setting it is possible to, e.g., encrypt ORAM content with a CPA-secure encryption scheme; see e.g. [GKW18]. While this remains possible in the full MPC setting, it becomes prohibitively expensive, due to the implied non-black-box use of cryptography. We focus on the latter more challenging setting where all memory inputs/outputs are simple additive secret shares, and all cryptography is black-box, even when we instantiate a client with an MPC protocol.*

The three main costs to consider in RAM-MPC are the per-access amortized bandwidth consumption (communication complexity), latency (round complexity), and computation. We can consider how these three costs scale as functions of the size of memory n and the bit-width of memory elements w . Of these three costs, computation is traditionally viewed as less important than the others, especially when the necessary computation can be parallelized and/or hardware-accelerated. Thus, we are often willing to increase computation cost if it means improving communication.

Trading Computation for Communication with Distributed Point Functions By using an appropriate off-the-shelf ORAM, e.g. Circuit ORAM [WCS15], it is possible to obtain a RAM-MPC protocol where all three costs scale linearly in w and at most polylogarithmically in n . While this is *far* preferable to compiling to linear scans, the polylogarithmic factors and moderate hidden constants of such an approach mean that memory accesses remain relatively expensive.

Of course, it is not necessary to use an *off-the-shelf* ORAM. The MPC setting is richer in its capabilities than the bare client/server ORAM setting, and we can leverage this fact to design a custom approach with lower cost. The literature has given many RAM-MPC constructions of this kind. Here, we discuss the two most relevant prior works; see Section 5 for discussion of other techniques.

One elegant example of a custom RAM-MPC is the *Floram* construction [Ds17]. In Floram, the two parties locally touch their shares of *each memory slot on each memory access*; thus, computation cost is linear. In exchange, Floram enjoys decent asymptotic communication cost and superb constant factors. Indeed, while Floram is asymptotically inferior to other off-the-shelf ORAMs, e.g. Circuit ORAM [WCS15], the authors found that their simple protocol achieved lower wall-clock runtime performance for a wide range of parameters.

In short, Floram leverages *distributed point functions* (DPFs, [GI14, BGI16]) to read/write to main memory in a secret-shared fashion; see Section 2 for a review of DPFs. As stated above, their Floram construction requires $\tilde{O}(wn)$ computation per access, but only $\tilde{O}(w\sqrt{n})$ bits of communication, where $\tilde{O}(\cdot)$ hides $\text{poly}(\lg n, \lambda)$ factors and where λ is a security parameter.

More recently, *Duoram* [VHG23] built on [Ds17]’s insight to achieve a RAM-3PC protocol with linear computation cost and—remarkably—only $O(w + \lambda \lg n)$ communication per access. Thus, for large enough words of size $w = \Omega(\lambda \lg n)$ bits, Duoram achieves *constant* communication overhead. Like Floram, Duoram also enjoys low constant factors and practically good performance. The authors also extend Duoram to the more challenging RAM-2PC setting, but to do so they leverage a particularly communication-efficient single-server private information retrieval (PIR) protocol called Spiral [MW22]. This PIR protocol builds on fully homomorphic encryption (FHE), and it is far less practical than the simple techniques [VHG23] use in the 3PC setting. Because of this use of PIR, their protocol cannot achieve constant communication overhead in the 2PC setting.

Single Access Machines; SAM-MPC Our goal is to further improve the asymptotic and concrete cost of semi-honest MPC handling of memory-manipulating programs. To do so, rather than directly targetting RAM programs, we build on the work of [AHR24] which proposed a weakening of RAM called the single access machine (SAM) model. The SAM model is essentially identical to the RAM model, except that each main memory slot may only be written¹ at most once, and read at most once.

Remark 2 (Compiling RAM to SAM). *While we formally work with the SAM model, our approach implies results for the more standard RAM model. This is because RAM programs straightforwardly compile to SAM programs. Namely, SAM allows to securely implement programs that manipulate tree-based data structures, so one can implement a random access memory as a search tree. This approach is reminiscent of the position map technique used in much of the ORAM literature; see Section 2.3 and [AHR24] for details. We choose to work with SAM rather than RAM both because our SAM results are more general, due to the above compilation strategy, and because the semantics of SAM are a natural match for our protocol.*

1.1 Contribution

2PC for SAM programs Following [Ds17, VHG23], we construct MPC handling of memory-manipulating programs on top of distributed point functions (DPFs). Our two-party protocol supports secure evaluation of arbitrary functions expressed as SAM programs. Our semi-honest 2PC protocol is built on only oblivious transfer (OT) and a black-box pseudorandom generator (PRG). The amortized cost of each memory access is as follows:

- **Communication:** The parties in an offline phase perform $O(\lg n)$ OTs on secrets of length λ , where λ is the length of a PRG seed. In the online phase, the parties exchange only $2 \lg n + 2w$ bits!
- **Rounds:** The offline phase, which generates correlated randomness for any polynomial number of accesses, takes $O(\lg n)$ rounds. In the online phase, each access takes 1 round.
- **Computation:** In both the offline and online phase, the parties handle each access in $\tilde{O}(wn)$ time. Offline and online computation is highly parallelizable.

Our protocol also supports parallel accesses. Any number of non-dependent memory accesses (i.e., the input to one access does not depend on the output of another) can be handled in one round.

¹In fact, our approach achieves a slightly stronger notion, where each memory cell may be written repeatedly; the cell accumulates an XOR of all values written to it.

	Model	Setting	Assumes	Offline		Online	
				Comm. (bits)	Comp.	Rounds	Comm. (bits) Comp.
OURS	SAM	2PC	OT	$O(\lambda \lg n)$	$\tilde{O}(wn)$	1	$2w + 2 \lg n$
	RAM			$O(\lambda(\lg^2 n / \lg B))$	$\tilde{O}(wn)$	$O(\lg n / \lg B)$	$2w + O(B(\lg^2 n / \lg B))$
[AHR24] via [WCS15]	SAM	2PC	OT	—	—	$\tilde{O}(1)$	$O(w\lambda)$
	RAM			—	—	$\tilde{O}(1)$	$O((w + \lg^2 n)\lambda)$
[ZWR ⁺ 16]	RAM	2PC	OT	—	—	$O(\lg n)$	$\tilde{O}(w\sqrt{n})$
	RAM	2PC	OT	—	—	$O(\lg n)$	$\tilde{O}(w\sqrt{n})$
[VHG23]	RAM	2PC	FHE	—	—	$O(1)$	$\tilde{O}(w)$
	RAM	3PC	OWF	$O(\lambda \lg n)$	$\tilde{O}(wn)$	$O(1)$	$\tilde{O}(wn)$
[JW18]	RAM	3PC	OWF	—	—	$O(\lg n)$	$O(\lambda \lg^3 n + w \lg n)$
	RAM	3PC	OWF	—	—	$O(1)$	$\tilde{O}(w\sqrt{n})$
[BKKO20]	RAM	2PC	OPRF	—	—	$O(1)$	$\tilde{O}(w\sqrt{n})$
	RAM	2PC	OPRF	—	—	$O(1)$	$\tilde{O}(w\sqrt{n})$
[HVO21]	SAM	2PC	OT	$O(w \lg n \lg \lg n \lambda)$	$\tilde{O}(w)$	0	0
	RAM			$O((w + \lg^2 n) \lg n \lg \lg n \lambda)$	$\tilde{O}(w)$		
[FNO24]	RAM	3PC	None	—	—	$O(\lg n)$	$O((w + \lg^2 n) \frac{\lg n}{\lg \lg n})$
	RAM						$\tilde{O}(w)$

¹ We list results for both SAM and RAM; see comparison of the models in Section 2.3.

² SAM operations `alloc/read/write` may incur different costs; we list the maximum of those costs.

⁴ w denotes word size (bits), n denotes memory capacity (words), and λ is a security parameter.

⁴ To implement RAM by SAM, one builds RAM recursively, instantiating a search tree; Parameter 2 $\leq B \leq n$ sets the tree's degree. As B increases, the tree depth (and hence round complexity) decreases; see also Section 2.3.

⁵ $\tilde{O}(\cdot)$ hides arbitrary $\text{poly}(\lg n, \lambda)$ factors. Therefore, for example, $\tilde{O}(1)$ should be considered inferior to $O(\lg n)$, but not inferior to $O(\sqrt{n})$.

⁶ When we do not list offline cost it is because the construction does not naturally asymptotically benefit from moving cost to an offline phase.

⁷ All protocols that list an offline phase can preprocess arbitrary numbers of accesses within $O(\lg n)$ rounds.

Table 1: Cost comparison of our approach with prior work.

Table 1 compares our approach to relevant prior work in terms of setting, cryptographic assumption, online round complexity, and online/offline communication/computation. Section 5 provides additional discussion of prior work.

We highlight that our 2PC SAM protocol is remarkably simple, easy to implement, and has excellent online communication cost.

We implemented the online phase of our protocol, instantiating the offline phase as a semi-honest untrusted third party that generates secret-shared permutations and DPF keys. Our implementation instantiates a PRG using hardware-accelerated AES. The code is available² online. Section 4 investigates our implementation’s performance.

Corollary: 2PC for pointer-machine programs and immutable memory programs [AHR24] showed that any *pointer-machine program* can be compiled to a SAM program [Sch80, KU58]. The pointer machine model is a natural model where the memory is structured as a graph, roughly akin to objects in an object-oriented programming language. A pointer-machine program may not use random access arrays, but it may use data structures based on explicit management of *pointers to objects*, e.g. stacks, queues, trees, graphs, etc.

[AH25] further showed that any program manipulating an *immutable memory*—i.e. where memory slots can be written to exactly once, then arbitrarily read—can also be compiled to a SAM program with constant overhead. This brings further programs into scope, including those written in purely functional programming languages (i.e. those based on Church’s foundational Lambda Calculus), as well as rich data structures like suffix trees and multidimensional range trees.

By combining our 2PC SAM protocol with the [AHR24, AH25] compilers, we obtain a 2PC protocol for pointer-machine programs and immutable memory programs. Such results were not known from prior techniques. In the following, we refer to the *in-degree* of an object, which is simply the number of existing copies of pointers to that object:

Corollary 1 (2PC for Constant-Degree/Immutable Pointer Machine Programs). *Assuming oblivious transfer, there exists a semi-honest 2PC protocol Π for any computation expressed as a pointer-machine program. If the pointer machine program runs in time n and if all objects are either (1) immutable or (2) have constant³ in-degree, then Π consumes $O(n \lg n)$ bits of online communication and $O(n \lg n)$ OT invocations.*

This result is remarkable in the sense that a *cleartext* constant in-degree pointer machine program will necessarily touch $O(n \lg n)$ bits (pointers are necessarily of size $O(\lg n)$ bits), so the online phase of the above protocol incurs constant communication overhead.

1.2 Technical Overview

Using DPFs to achieve PIR write The main building block of our protocol is that of a “secret-shared PIR write”. Namely, the parties hold a secret-shared index α and a secret-shared value β , and they wish to construct a secret-shared length- n array that holds zero everywhere, except at index α , where the array holds β . This building block can be efficiently instantiated from a random, preprocessed DPF and $2w + 2 \lg n$ bits of online communication, where w is the bit-length of array elements; see Section 2 for details.

Protocol Sketch With this PIR write mechanism in hand, our basic 2PC SAM protocol is informally as follows: To write a secret-shared value to memory, the parties use the above PIR write to efficiently and obliviously save secret shares of that value to a randomly chosen (without replacement) memory slot. To read secret shares of the value back later, the parties reveal to themselves the randomly chosen memory slot in the clear, then fetch their secret shares. This revelation is secure because (1) the location is random and (2) the SAM model ensures that we will read each location at most once. At least at a high level, this is the entire protocol.

²<https://github.com/DAHeath/2pc-sam>

³If the pointer-machine program manipulates mutable objects with higher in-degree, then each time it reads a pointer to an object with in-degree d , it incurs $O(\lg n \lg d)$ bits of online communication [AHR24].

The simplicity of this protocol leads to its low cost: Each memory write consumes a single DPF, and each memory read is almost free. DPFs can be prepared in the offline phase, and in the online phase, each memory operation consumes only a small number of bits.

Despite the simplicity of this protocol, to our knowledge it has not been observed by prior work, possibly because it naturally instantiates SAM—not RAM—and SAM was only recently formalized [AHR24]. It is worth pointing out that when we use the above simple protocol to instantiate RAM (by compiling the RAM program to a SAM program; see Section 2.3), this basic approach improves over prior work Floram [Ds17] by reducing asymptotic overhead and over prior work Duoram [VHG23] by removing the expensive use of single-server PIR.

2 Preliminaries

2.1 Notation

- $\lg$ denotes the base-2 logarithm. Namely, $\lg x \triangleq \lceil \log_2 x \rceil$.
- We consider a two-party protocol running between parties P_0 and P_1 .
- n denotes the size of memory and w denotes word size. For simplicity, we assume n is a power of two.
- In general, we use $\alpha \in \{0,1\}^{\lg n}$ to denote a memory address and $\beta \in \{0,1\}^w$ to denote a memory value.
- λ denotes a security parameter (e.g., the length of PRG seeds).
- $x \triangleq y$ denotes that x is equal to y by definition.
- $x \approx y$ denotes that ensembles x and y are computationally indistinguishable (in λ).
- Let $x \in \{0,1\}^*$ denote a string. $\llbracket x \rrbracket$ denotes a two-party XOR sharing of x . We denote P_0 's share by $\llbracket x \rrbracket_0$ and P_1 's share by $\llbracket x \rrbracket_1$. Namely, $\llbracket x \rrbracket_0 \oplus \llbracket x \rrbracket_1 = x$.
- We sometimes use $\tilde{O}(\cdot)$ notation, especially for computation costs. This notation hides $\text{poly}(\lg n, \lambda)$ factors.

2.2 Security Model and Functionalities

Security Model Our protocol is secure against a semi-honest adversary who corrupts one of the two parties. We specifically consider semi-honest security as defined by Canetti [Can00] as a special-case of UC security. Namely, we consider an adversary $\mathcal{A}$ who can read the corrupted party's state. The goal in showing security is to produce a simulator $\mathcal{S}$ that—based on an interaction with the ideal functionality—can emulate the behavior of $\mathcal{A}$. Indistinguishability of the real-world adversary and ideal-world simulator demonstrates that the protocol leaks nothing beyond what is implied by the ideal functionality.

This definition of semi-honest security is slightly more general than the more typical one where the simulator needs only produce a protocol transcript indistinguishable from the real-world view of the corrupted party; see e.g. [Lin16]. We consider this slightly more general definition to construct a modular *reactive* functionality with a more efficient instantiation than would have been possible with the more standard notion of semi-honest security. We explain:

Looking ahead, our functionality implements the modular reactive interface of a single-access memory, essentially allowing parties to read/write an array. In our *protocol*, parties perform a read by simply fetching and outputting secret shares already held in their local storage. Because the shares were already in the corrupted party's local storage, our functionality cannot, e.g., say that the shares returned by read are fresh, uniform secret shares – this would simply be false. If we wanted fresh uniform secret shares, the parties would need to exchange more bits, which is undesirable. Instead, the functionality asks the ideal-world adversary (i.e., the simulator) to specify secret shares. At an intuitive level, this roughly models the idea that a real-world adversary can “remember” secret shares it already saw. This approach of having the adversary specify shares/randomness is common in the malicious model, see e.g. [MR19].

Functionality $\mathcal{F}_{\text{pre}}$:

- **Preprocess distributed point function:** Upon receiving $(\text{dpf}, \lg n, m)$ from both parties:
 - Uniformly sample $\alpha \in_{\$} \{0, 1\}^{\lg n}, \beta \in_{\$} \{0, 1\}^m$.
 - Call $(k_0, k_1) \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta)$. Send k_0 to P_0 , and send k_1 to P_1 .
 - Distribute additive shares $\llbracket \alpha \rrbracket, \llbracket \beta \rrbracket$ to the two parties. Shares of α, β are chosen arbitrarily; if a party is corrupted, then the adversary chooses output shares for the corrupted party, and $\mathcal{F}_{\text{sam}}$ sets honest party shares accordingly.
- **Preprocess permutation:** Upon receiving $(\text{permute}, n)$, uniformly sample a permutation π on n elements, where π is encoded as a bitstring of length $n \lg n$. Sample a uniform secret sharing $\llbracket \pi \rrbracket$, and send one share to each party. Shares of π are chosen arbitrarily; if a party is corrupted, then the adversary chooses output shares for the corrupted party, and $\mathcal{F}_{\text{sam}}$ sets honest party shares accordingly.

Figure 1: We build secure two-party execution of SAM programs on the above preprocessing functionality $\mathcal{F}_{\text{pre}}$. In the 2PC setting, DPFs can be instantiated from OT and a black-box PRG [Ds17], and permutations can be instantiated via a generic Boolean 2PC protocol and two invocations of permutation network [Wak68].

We emphasize that an outer protocol that builds on top of our ideal-world functionality can ultimately satisfy the simpler notion of semi-honest security.

Functionalities We build our SAM protocol assuming access to a preprocessing functionality $\mathcal{F}_{\text{pre}}$; see Figure 1. Our preprocessing functionality $\mathcal{F}_{\text{pre}}$ allows parties to sample DPF keys (see review of DPFs in Section 2.5) and secret-shared uniform permutations. $\mathcal{F}_{\text{pre}}$ can be instantiated in the 2PC setting in to OT-hybrid model given black-box access to a PRG [Ds17]. Preprocessing a permutation of size n can be achieved by a generic OT-based 2PC protocol for Boolean circuits. This protocol consumes $O(n \lg n)$ OTs; we explain in more detail below. Thus, the whole preprocessing protocol only requires OT and a black-box PRG, and OT can be made efficient with the help of efficient OT extension, e.g. [IKNP03, BCG⁺19, Roy22].

Our goal is to securely instantiate the ideal functionality $\mathcal{F}_{\text{sam}}$; see Figure 2. In short, $\mathcal{F}_{\text{sam}}$ allows parties to read/write a secret-shared memory, with restrictions placed on reads. Formally, the parties may (1) initialize a fresh SAM, (2) allocate memory addresses in that SAM, (3) read an allocated address, and (4) write to an allocated address. In our functionality, writing the same address accumulates an XOR of all written values at the specified address. Reading the same address twice is considered an error, and we only consider SAM programs that do not cause errors. We review the SAM model further in Section 2.3.

In our handling of allocate/read, we allow the semi-honest adversary to choose its output secret shares, and the functionality sets honest party shares accordingly. Our protocol can securely achieve this notion because these shares are already stored locally at the parties; see also our proof of security (Section 3.2).

Preprocessing a shared permutation π In our handling of SAM, the parties allocate memory addresses according to a secret permutation π ; it is crucial that neither party knows π . This permutation is data-independent, so it can be computed in the offline phase. Our preprocessing functionality $\mathcal{F}_{\text{pre}}$ allows the parties to sample such a secret-shared permutation.

It is well known that in the 2PC setting, a secret-shared random permutation π can be computed via two invocations of a Waksman or Beneš *permutation network* [Wak68, Ben64]. Namely, the parties compute a permutation $\pi = \pi_0 \circ \pi_1$ where party P_i knows π_i , but not π_{1-i} . When subpermutations π_i are uniform, so is π , and neither party knows π . A Waksman network on n inputs can be computed by a circuit with $n \lg n - n + 1$ *swap* gates on $(\lg n)$ -bit words. Thus, this step can be handled by $O(n \lg n)$ 1-out-of-2 OTs on strings of length $\lg n$.

Functionality $\mathcal{F}_{\text{sam}}$:

- **Parameters:** $\mathcal{F}_{\text{sam}}$ is parameterized by word-size w and array-size n .
- **Initialization:** At initialization, $\mathcal{F}_{\text{sam}}$ creates a fresh size- n array A storing w -bit words. Each array slot $A_\alpha \leftarrow 0^w$ is initialized as the all-zeros string.
- **Commands:** $\mathcal{F}_{\text{sam}}$ responds to the following commands:
 - **Allocate:** Upon receiving **alloc** from both parties, uniformly sample a previously-unallocated address $\alpha \in [n]$, and respond with shares $\llbracket \alpha \rrbracket$. Shares are chosen arbitrarily; if a party is corrupted, then the adversary chooses the output share for the corrupted party, and $\mathcal{F}_{\text{sam}}$ sets the honest party's share such that shares sum to α . If $\mathcal{F}_{\text{sam}}$ already received n commands **alloc**, then this command is considered **invalid**.
 - **Write:** Upon receiving from the parties **(write, $\llbracket \alpha \rrbracket$, $\llbracket \beta \rrbracket$)** for $\beta \in \{0, 1\}^w$, set $A_\alpha \leftarrow A_\alpha \oplus \beta$.
 - **Read:** Upon receiving from the parties **(read, $\llbracket \alpha \rrbracket$)**, look up $\beta \leftarrow A_\alpha$, send α to the adversary, and respond with shares $\llbracket \beta \rrbracket$. Shares of β are chosen arbitrarily; if a party is corrupted, then the adversary chooses the output share for the corrupted party, and $\mathcal{F}_{\text{sam}}$ sets the honest party's share such that shares sum to β . If $\mathcal{F}_{\text{sam}}$ already received a command **(read, $\llbracket \alpha \rrbracket$)** for the same address α , then this command is considered **invalid**.

A sequence of $\mathcal{F}_{\text{sam}}$ commands is considered **valid** if no command in the sequence is **invalid**.

Figure 2: Our target two-party SAM functionality $\mathcal{F}_{\text{sam}}$ implements a secret-shared array where each array slot may be read at most once. Repeatedly writing to an array slot accumulates the writes via XOR.

2.3 The Single Access Machine (SAM) Model

Seeking improvement to oblivious algorithms and data structures, [AHR24] introduced SAM as a weakening of the RAM model of computation. Because SAM is more restrictive on the memory-manipulating program than RAM, it can be easier to implement SAM-based memory. In particular, [AHR24] observed that certain ORAM techniques naturally achieve SAM. The authors took off-the-shelf ORAM constructions Path ORAM [SvDS⁺13] and Circuit ORAM [WCS15] and transformed them to achieve a notion of Oblivious SAM (OSAM). This transformation involved *removing* a large component—the so-called position map—of those constructions, reducing their asymptotic complexity.

Our contribution broadens the set of techniques that naturally implement SAM. Note that our approach to implementing SAM is entirely different than that of [AHR24], as it leverages DPFs to implement SAM.

Usefulness of SAM Limitations placed on SAM programs may at first glance seem overly restrictive, but [AHR24] showed that the SAM model is surprisingly expressive. In short, any pointer machine program (i.e., where the program can read/write pointers, but cannot, e.g., add a constant to a pointer) can be compiled to a SAM program. The cost of this compilation is somewhat nuanced: each time the pointer program dereferences a memory cell with d incoming pointers, amortized $O(\lg d)$ SAM operations are consumed.

In many cases, the authors show it is possible to keep the number of pointers to any memory cell to a constant, thus achieving *no* asymptotic blow-up. For example, stacks, queues, doubly-linked lists, trees, and even low-degree graphs are efficiently instantiable in the SAM model. Indeed, even graph traversals over arbitrary-degree graphs can be achieved without asymptotic blow-up [AHR24]. [AH25] further extended the set of programs achievable from SAM, showing that many interesting algorithms on strings and even programs written in purely functional programming languages efficiently compile to a SAM program. Thus, our new SAM protocol implies new results for the cost of running a variety of algorithms inside MPC.

As one example, consider performing a graph traversal with depth-first or breadth-first search (DFS or BFS). DFS/BFS on a graph of size n (with word size $w = \Theta(\lg n)$) can be achieved by $O(n)$ SAM operations [AHR24], so our protocol implies a privacy-preserving algorithm for graph traversal that incurs only $O(n \lg n)$ bits of online communication. Thus, the online communication complexity of the 2PC protocol matches the total number of bits touched in a cleartext execution of the same algorithm. Contrast this with

standard ORAM-based execution of the same algorithm using Circuit ORAM [WCS15], where online cost would be $O(n \lg^2 n \lambda)$ bits.

Of course, while we implement SAM, one might also be interested in how our protocol compares with prior work on RAM. Fortunately, any RAM program can be compiled to a SAM program with small blow-up; we discuss shortly.

Managing SAM addresses A SAM program may allocate a memory address, then read/write⁴ that address later on. But in the SAM model, the client program should run in constant space and offload memory storage to the single access memory, so how can the client program remember addresses it allocated earlier? The answer is that the client program may allocate an address, then save that address into SAM memory such that it can be read back and used later. It is in this way that the client program can build up tree-like and graph-like structures: a node in a binary tree is encoded as a pair of addresses to its children, and this pair can be saved as a memory word; see [AHR24] for more details.

2.4 How to Implement RAM with SAM

As discussed above, the SAM model supports programs that manipulate trees. Because of this, it is straightforward to implement RAM with SAM [AHR24]. Namely, we can store the content of RAM in a balanced trie where each i -th leaf of the trie stores to the i -th slot of RAM. Reading/writing the RAM involves walking to the appropriate leaf node.

In many ORAMs, the client maintains a *position map*, which maps array indices to positions in the ORAM structure. Typically, this map is implemented recursively. The above technique of implementing RAM as a tree essentially instantiates the recursive position map as a single tree structure. This arrangement places the data words as well as all position-map metadata into a single SAM memory.

Using large words to reduce RAM overhead Here, we discuss and work out the details of an alternative arrangement of SAM-based RAM, which was not discussed in [AHR24]. We leave this discussion in preliminaries because the techniques are either known (in the setting of position maps) or straightforward.

As an alternative to storing the position map and data words in a single SAM, one can store the data words and the corresponding metadata in two⁵ separate SAMs. The advantage of this is that we can choose different word sizes for the two SAMs. To read a RAM memory element, we first traverse a trie path through the metadata SAM to find the SAM address of the desired memory element, then we read that address from the data SAM. Note that the interleaved operations on two SAMs does not pose a security problem because the metadata trie is balanced; for each RAM access, the parties observe a fixed-length sequence of $O(h)$ operations on the metadata SAM, where h is the height of the RAM trie, then $O(1)$ operations on the data SAM.

This alternative is particularly attractive if the top-level RAM stores large words, e.g. if $w = \Omega(\lg^2 n)$, because it allows us to erase the online asymptotic communication overhead of the recursive position map. Namely, if the RAM trie has depth $O(\lg n)$, and if the metadata ORAM stores words of size $w' = O(\lg n)$, then the total number of bits read from SAM to traverse a path is only $O(\lg^2 n)$, asymptotically matched or exceeded by the size of a data word. In the literature and in the context of standard ORAM position maps, this is sometimes called the big block/little block trick; see e.g. [SvDS⁺13, WCS15].

In our approach, reading/writing SAM incurs constant rounds of communication, so when implementing RAM we incur a number of rounds that scales with the length of paths through the RAM trie. So if we choose to implement RAM as a binary trie, then we use $O(\lg n)$ rounds of communication per access.

If RAM memory words are large, we can also improve round complexity of RAM, without changing the asymptotic communication complexity. Namely, we can choose a parameter $2 \leq B \leq n$, and set this as the degree of our RAM trie. This reduces the height of the RAM trie to only $O(\lg n / \lg B)$. If $B = O(w / \lg^2 n)$, then it this incurs no increase to the asymptotic communication complexity. This reduction in height in

⁴A SAM program may read an address that has not been written, in which case the SAM must return some default value. This capability is important for using SAM to implements handling of DAG-like structures [AHR24]. Our 2PC SAM returns the all zeros string when reading an unwritten value.

⁵Or, if the position map has L levels, we can use L SAMs. We discuss here how to achieve the result with two SAMs because further decomposing to L SAMs is more complex and improves performance by (usually much) less than $2\times$.

turn decreases the number of needed SAM operations by factor $\lg B$. If RAM words are *very* large, this can even reduce round complexity to a constant. In particular, suppose that $w = \Omega(\lambda \lg^2 n)$, and suppose that $n = O(\lambda^c)$ is bounded by some fixed polynomial in the security parameter, as is relatively standard. In this case, we can set $B = \lambda$ without blowing up communication complexity, thus achieving a tree of depth $O(\lg n / \lg \lambda) = O(1)$. Thus, for very large word size, our approach achieves 2PC-RAM where each RAM access incurs constant communication overhead and constant rounds.

2.5 Distributed Point Functions

We review the concept of a distributed point function (DPF) [GI14, BGI16], which is central to our approach.

A DPF is a function secret sharing scheme (FSS [BGI15]) for the class of point functions:

Definition 1 (Point Function). *Let $\alpha \in \{0, 1\}^{\lg n}$ denote an index and let $\beta \in \{0, 1\}^m$ denote a value. The **point function that stores β at position α** is a function $\text{Point}(\alpha, \beta, \cdot) : \{0, 1\}^{\lg n} \rightarrow \{0, 1\}^m$ defined as follows:*

$$\text{Point}(\alpha, \beta, x) \triangleq \begin{cases} \beta & \text{if } x = \alpha \\ 0 & \text{otherwise} \end{cases}$$

For simplicity, we consider a restriction of DPFs with Boolean domain/range. More generally, DPFs can range over arbitrary finite groups, but we do not need this flexibility for our approach.

Definition 2 (Distributed Point Function (DPF)). *A function secret sharing scheme for two-party **distributed point functions** is a pair of algorithms $(\text{Gen}, \text{Eval})$, defined as follows:*

- $\text{Gen}(1^\lambda, \alpha, \beta)$ takes as input $\alpha \in \{0, 1\}^{\lg n}, \beta \in \{0, 1\}^m$; Gen outputs a pair of keys k_0, k_1 .
- $\text{Eval}(k, x)$ takes as input a key generated by Gen and an index $x \in \{0, 1\}^{\lg n}$. It outputs m bits.

The scheme is **correct** if evaluations $\text{Eval}(k_0, x), \text{Eval}(k_1, x)$ yield XOR shares of the specified point function. Let $(k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \alpha, \beta)$. Then the following must hold:

$$\text{Eval}(k_0, x) \oplus \text{Eval}(k_1, x) = \text{Point}(\alpha, \beta, x)$$

The scheme is **private** if there exists a simulator $\mathcal{S}_{\text{dpf}}$ such that for every $b \in \{0, 1\}$ and for every polynomial-size point function description $\alpha \in \{0, 1\}^{\lg n}, \beta \in \{0, 1\}^m$, the following computational indistinguishability (in λ) holds:

$$\{ k_b \text{ where } (k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \alpha, \beta) \} \approx \{ \mathcal{S}_{\text{dpf}}(1^\lambda, b) \}$$

While efficient constructions for generating DPF keys exist, such constructions require a dealer [BGI16]. In the 2PC setting, such a dealer is not available, so we need to instantiate such a dealer by means of a protocol; [Ds17] gave a suitable construction:

Fact 1 (2PC DPF Protocol [Ds17]). *Assuming PRGs exist and in the OT-hybrid model, there exists a 2PC protocol for generating DPF keys from $\{0, 1\}^{\lg n}$ to $\{0, 1\}^m$ that consumes $O(\lg n)$ 1-out-of-2 OTs over λ -bit strings, and $O(m)$ additional bits of communication. The parties expend $\tilde{O}(nm)$ local computation.*

Derandomizing Distributed Point Functions In our protocol, we generate all DPF keys in the offline phase. These preprocessed keys are for a randomly chosen point function, and in the online phase we derandomize them. Here, we explain how this derandomization is achieved.

Fact 2 (Shifting DPFs). *There exists a local procedure **Shift** that satisfies the following correctness condition. Let $\alpha, \alpha' \in \{0, 1\}^{\lg n}$, let $\beta, \beta' \in \{0, 1\}^m$, and let $(k_0, k_1) \leftarrow \text{Gen}(1^\lambda, \alpha, (\beta, 1))$. Then:*

$$\text{Shift}(k_0, \alpha', \beta', x) \oplus \text{Shift}(k_1, \alpha', \beta', x) = \text{Point}(\alpha \oplus \alpha', \beta \oplus \beta', x)$$

Here, the pair $(\beta, 1)$ denotes the $(m+1)$ -bit concatenation of β with a one at the end.

Proof. The key detail here is that we pad the output of the DPF with an extra bit 1, and we use this to adjust the DPF’s output value.

More precisely, to adjust the evaluation point α by distance α' , the parties simply evaluate the underlying DPF key at point $x \oplus \alpha'$ rather than at point x . Now, the parties hold the following by correctness of the DPF:

$$\begin{cases} (\llbracket \beta \rrbracket, \llbracket 1 \rrbracket) & \text{if } x = \alpha \oplus \alpha' \\ (\llbracket 0^m \rrbracket, \llbracket 0 \rrbracket) & \text{otherwise} \end{cases}$$

That is, they obtain at the point $\alpha \oplus \alpha'$ a secret share of the DPF payload and an extra 1. They locally scale their shares of this shared 1 by β' , then add it to the payload:

$$\begin{cases} \llbracket \beta \rrbracket \oplus (\beta' \cdot \llbracket 1 \rrbracket) & \text{if } x = \alpha \oplus \alpha' \\ \llbracket 0^m \rrbracket \oplus (\beta' \cdot \llbracket 0 \rrbracket) & \text{otherwise} \end{cases} = \begin{cases} \llbracket \beta \oplus \beta' \rrbracket & \text{if } x = \alpha \oplus \alpha' \\ \llbracket 0^m \rrbracket & \text{otherwise} \end{cases} = \llbracket \text{Point}(\alpha \oplus \alpha', \beta \oplus \beta', x) \rrbracket$$

Thus, **Shift** is correct. □

The crucial takeaway from Fact 2 is that if we wish to construct a DPF with specified evaluation point α and value β , it suffices to preprocess a *random* DPF with uniform evaluation point α' and uniform value β' . Given secret shares of $\alpha, \alpha', \beta, \beta'$, the parties can compute secret shares of $\alpha \oplus \alpha'$ and $\beta \oplus \beta'$, reveal these differences to themselves (which is secure because α' and β' are uniform), then locally call **Shift** on their respective keys.

Improving DPF computation overhead As discussed so far, the method by which two parties evaluate a DPF on its full size- n domain is by calling **Eval** (or **Shift**) n times. Existing PRG-based DPFs offer a better procedure for this “full domain” evaluation, which reduces the total number of PRG calls by factor $O(\lg n)$. In short, PRG-based DPFs involve using the PRG to induce a GGM-style binary tree [GGM84], and evaluation involves walking a path through that tree. A full tree walk on a tree of size n is $O(\lg n)$ times more efficient than n times traversing a path through the same tree; see [BG116] for details. Our implementation applies this optimization.

3 Technical Approach

3.1 Protocol Details

For a brief informal explanation of our approach, see Section 1.2. Figure 3 lists our formal protocol. Our protocol handles each SAM operation as follows:

- $\alpha \leftarrow \text{Alloc}$: Parties allocate memory addresses in a secret, random order. Namely, the sequence of allocations is described by a uniform permutation π , and each i -th memory address α_i is π_i . π is sampled during preprocessing.
- $\text{Write}(\alpha, \beta)$: Parties use a DPF to construct a secret-shared vector where index α holds β and all other indexes hold 0; they element-wise XOR this vector into the memory, writing β to address α .
- $\beta \leftarrow \text{Read}(\alpha)$: Parties exchange shares to reveal α . They locally read respective shares of memory address α .

The approach is correct by construction: each write saves element β to address α by the correctness of DPFs, so reading address α in the clear correctly fetches the element written there. The only challenge in terms of security is that on each read the parties reveal an address to themselves. But when SAM addresses are used correctly by a calling protocol, these revelations can be simulated because (1) addresses are uniformly permuted and (2) the SAM model guarantees each address is read at most once. Thus, each revealed address appears uniformly random (without replacement). See Section 3.2 for our security proof.

Protocol Π_{sam} :

- **Initialize:** Parties use $\mathcal{F}_{\text{pre}}$ to preprocess a size- n uniform, secret-shared permutation $\llbracket \pi \rrbracket$.

Parties also use $\mathcal{F}_{\text{pre}}$ to preprocess n uniform DPFs from $\{0, 1\}^{\lg n}$ to $\{0, 1\}^{w+1}$. w bits of the range are for storing a data value, 1 bit is for the purpose of derandomization (see Section 2.5). In particular, for each $i \in [n]$ the preprocessing functionality (1) samples uniform $\alpha'_i \in \{0, 1\}^{\lg n}$, (2) samples uniform $\beta'_i \in \{0, 1\}^w$, and (3) generates DPF keys as follows:

$$k_{0,i}, k_{1,i} \leftarrow \text{Gen}(1^\lambda, \alpha', (\beta', 1))$$

$\mathcal{F}_{\text{pre}}$ distributes $\llbracket \alpha'_i \rrbracket$ and $\llbracket \beta'_i \rrbracket$ to the parties, sends $k_{0,i}$ to P_0 , and sends $k_{1,i}$ to P_1 .

Parties initialize a secret-shared memory state $\llbracket \sigma \rrbracket$. Namely, σ is a length- n all-zeros array, where each σ_i is w bits long.

- **Allocate:** Let this be the i -th allocation. Parties output secret shares of the i -th entry of the preprocessed permutation: $\llbracket \alpha \rrbracket \triangleq \llbracket \pi_i \rrbracket$.
- **Write:** Let this be the i -th write. Parties input secret-shared address $\llbracket \alpha \rrbracket$ and value $\llbracket \beta \rrbracket$. Parties fetch their i -th preprocessed DPF corresponding to random index α'_i and value β'_i :

$$k_{0,i}, k_{1,i}, \llbracket \alpha'_i \rrbracket, \llbracket \beta'_i \rrbracket$$

Parties locally compute $\llbracket \alpha \oplus \alpha'_i \rrbracket$ and $\llbracket \beta \oplus \beta'_i \rrbracket$, then exchange shares to reveal $\alpha \oplus \alpha'_i$ and $\beta \oplus \beta'_i$. This exchange consumes 1 round and $2 \lg n + 2w$ bits. Recall from Section 2.5 that parties can derandomize their DPF shares at each index $j \in \{0, 1\}^{\lg n}$ as follows:

$$(\text{Shift}(k_{0,i}, \alpha \oplus \alpha'_i, \beta \oplus \beta'_i, j), \text{Shift}(k_{1,i}, \alpha \oplus \alpha'_i, \beta \oplus \beta'_i, j)) = \llbracket \text{Point}(\alpha, \beta, j) \rrbracket$$

Parties use their derandomized DPF to update each index j of the state:

$$\llbracket \sigma_j \rrbracket \leftarrow \llbracket \sigma_j \rrbracket \oplus \llbracket \text{Point}(\alpha, \beta, j) \rrbracket$$

This correctly updates index α to hold value β , and it has no effect on other indices.

- **Read:** Parties input a secret share $\llbracket \alpha \rrbracket$. They exchange shares, revealing α in the clear. This exchange consumes 1 round and $2 \lg n$ bits. Parties look up index α of the secret-shared array and output $\llbracket \sigma_\alpha \rrbracket$.

Figure 3: Our protocol Π_{sam} realizes the $\mathcal{F}_{\text{sam}}$ functionality for valid sequences of commands (Figure 2) in the $\mathcal{F}_{\text{pre}}$ -hybrid model (Figure 1). Roughly, parties allocate memory addresses in a permuted order; to write, they use a DPF; to read, they open the read address and read their local memory. It is safe to reveal read addresses because (1) addresses are permuted and (2) each address is read at most once.

We note that our SAM functionality places the burden of managing SAM addresses on the calling protocol. Formally, this manifests in the fact that $\mathcal{F}_{\text{sam}}$ reveals read addresses α to the adversary. Roughly, as long as the calling protocol uses SAM addresses as intended, it is straightforward to prove security.

In more detail, our SAM functionality cannot enforce that a calling protocol does not use SAM addresses in a way that breaks security. For instance, a calling protocol might ignore output of the alloc procedure and make up its own addresses, or it might mangle SAM addresses by running procedures on their bits, and so on. As long as a calling procedure (1) allocates all SAM addresses by calling alloc and (2) does not change the content of an address or conditionally branch its behavior depending on the bits of an address, then the outer protocol’s usage of SAM is straightforwardly secure: Every allocated SAM address is secret-shared and uniform, and it is revealed at most once when it is eventually read. Thus, while the adversary in the outer protocol is given addresses α on each call to read, these are trivially simulated by uniformly sampling without replacement.

Cost Accounting Consider a sequence of $O(n)$ SAM operations on a memory of size n and with w -bit words.

- **Offline communication.** The bottleneck in offline communication is from preprocessing DPFs, which incurs $O(\lambda \lg n)$ bits per **Write**.
- **Offline rounds.** All DPFs can be preprocessed in parallel, and Waksman’s network has depth $O(\lg n)$, so the total round complexity of the offline phase is only $O(\lg n)$ rounds.
- **Offline computation.** [Ds17]’s DPF protocol runs in $\tilde{O}(wn)$ time, which is the offline computation bottleneck.
- **Online communication.** The most expensive operation is **Write**, which requires derandomization of a DPF, incurring $2w + 2 \lg n$ bits of communication.
- **Online rounds.** Each operation type finishes within 1 round.
- **Online computation.** Derandomizing and evaluating a DPF at all indices takes $\tilde{O}(wn)$ time.

3.2 Security Proof

Theorem 1. *Assuming the parties issue a sequence of **valid** SAM commands (Figure 2), Π_{sam} realizes $\mathcal{F}_{\text{sam}}$ with semi-honest security in the $\mathcal{F}_{\text{pre}}$ -hybrid model.*

Proof. By constructing a simulator for each party P_i .

The protocol is symmetric, so without loss of generality, assume P_0 is corrupt, and let $\mathcal{S}$ be P_0 ’s simulator. In the semi-honest model, the simulator $\mathcal{S}$ interacts with the ideal functionality $\mathcal{F}_{\text{sam}}$ and the hybrid-world adversary $\mathcal{A}$. The proof goal is to show that the output of $\mathcal{S}$, together with the output of an ideal-world honest party P_1 , is indistinguishable from P_0 ’s real-world view, together with the output of real-world honest party P_1 . In other words, we wish to show we can simulate P_0 ’s view, in the context of the honest party’s output.

Our simulator $\mathcal{S}$ forges a convincing protocol transcript by (1) playing the role of the preprocessing functionality $\mathcal{F}_{\text{pre}}$ and (2) keeping track of simulated party states. Precisely, $\mathcal{S}$ handles each of the four SAM operations as follows:

- **Initialize:** $\mathcal{S}$ simulates initialization by playing the preprocessing functionality. $\mathcal{S}$ first samples a size- n array of $(\lg n)$ -bit strings R , where each entry R_i is uniform. R is meant to act as P_0 ’s secret share of the uniformly random permutation π . It attaches R to P_0 ’s simulated view. $\mathcal{S}$ n times runs the DPF simulator $\mathcal{S}_{\text{dpf}}$. For each $i \in [n]$, it calls:

$$k_{0,i} \leftarrow \mathcal{S}_{\text{dpf}}(1^\lambda, 0)$$

It attaches these DPF keys to P_0 ’s simulated view. In addition, for each $i \in [n]$, $\mathcal{S}$ must sample “shares” of random address α'_i and random address β'_i . Due to the structure of the preprocessing functionality,

the hybrid-world adversary will at this point try to send its shares of α'_i, β'_i to the $\mathcal{F}_{\text{pre}}$ functionality. $\mathcal{S}$ simply collects and attaches these to the simulated view. Recall that in the protocol, the parties maintain a secret-shared state $\llbracket \sigma \rrbracket$. To mirror this, $\mathcal{S}$ also initializes an all-zeros array σ_0 that will simulate P_0 's share of the state.

- **Allocate:** $\mathcal{S}$ fetches the i -th entry of the simulated permutation share R_i . Recall that on the allocate operation, the ideal functionality outputs a secret sharing, and it allows the adversary (i.e. $\mathcal{S}$) to choose its share. $\mathcal{S}$ sends `alloc` to $\mathcal{F}_{\text{sam}}$, choosing as its output share R_i . Accordingly, the functionality chooses a uniformly random address (without replacement) α_i , and it sends $R_i \oplus \alpha_i$ to the honest party P_1 .
- **Write:** For a write operation, $\mathcal{S}$ holds input shares $\llbracket \alpha \rrbracket_0, \llbracket \beta \rrbracket_0$ and P_1 holds $\llbracket \alpha \rrbracket_1, \llbracket \beta \rrbracket_1$. $\mathcal{S}$ and P_1 send their respective shares to the ideal functionality $\mathcal{F}_{\text{sam}}$. The ideal functionality updates $A_\alpha \leftarrow A_\alpha \oplus \beta$. Let this be the i -th write. Recall that in the protocol, the parties exchange shares to reveal $\alpha \oplus \alpha'_i$ and $\beta \oplus \beta'_i$. But α'_i and β'_i are uniform, so $\mathcal{S}$ simulates the shares sent by P_1 via uniform strings, and it attaches those to P_0 's simulated view. Let $\tilde{\alpha}$ (resp. $\tilde{\beta}$) denote the simulated revealed address difference (resp. simulated revealed value difference). $\mathcal{S}$ updates each index j of the simulated state share σ as follows:

$$\sigma_j \leftarrow \sigma_j \oplus \text{Shift}(k_{0,i}, \tilde{\alpha}, \tilde{\beta}, j)$$

This update to σ reflects the change to P_0 's local share of state in the protocol.

- **Read:** For a read operation, $\mathcal{S}$ holds input share $\llbracket \alpha \rrbracket_0$. It sends (`read`, $\llbracket \alpha \rrbracket_0$) to the ideal functionality $\mathcal{F}_{\text{sam}}$. The honest party P_1 sends (`read`, $\llbracket \alpha \rrbracket_1$) to $\mathcal{F}_{\text{sam}}$. Recall that $\mathcal{F}_{\text{sam}}$ sends the address α to the adversary (i.e., $\mathcal{S}$) in the clear, so $\mathcal{S}$ receives α from the functionality. Recall also that on a read $\mathcal{F}_{\text{sam}}$ outputs a secret sharing, and it allows the adversary to choose its share. $\mathcal{S}$ fetches from its simulated state σ_α and chooses this value as its output share. Accordingly, the ideal functionality fetches A_α and sends $\sigma_\alpha \oplus A_\alpha$ to the honest party P_1 . $\mathcal{S}$ computes and attaches to P_0 's simulated view the particular value $\llbracket \alpha \rrbracket_0 \oplus \alpha$, consistent with the real-world message that would reveal address α to P_0 .

Claim: The following two quantities are indistinguishable:

- **Ideal** P_0 's simulated view paired with P_1 's output.
- **Real** P_0 's view in Π_{sam} paired with P_1 's output from Π_{sam} .

Indeed, this essentially follows by construction of the above simulator. We note the important points of comparison:

- **DPF keys:** In the real world, P_0 obtains a DPF key from $\mathcal{F}_{\text{pre}}$; in the ideal world, P_0 obtains simulated DPF keys from the simulator. But those DPF keys are generated by $\mathcal{S}_{\text{dpf}}$, and Definition 2 states that real keys are indistinguishable from simulated keys (note that P_1 's keys never appear in P_0 's view).
- **Allocate:** In our protocol, allocation is a local operation, so simulation is trivial.
- **Write:** In both worlds, P_0 observes two uniformly random strings incoming from P_1 , one for a difference in addresses, one for a difference in values.
- **Read:** In both worlds, P_0 receives from P_1 a share consistent with an address that has never been read before.

Thus, the real and ideal worlds are indistinguishable, so Π_{sam} indeed realizes the ideal functionality $\mathcal{F}_{\text{sam}}$. $\square$

4 Evaluation

This section evaluates the concrete efficiency of our approach by experiments and analysis. In this section by a SAM “access”, we refer to an allocation and a pair of a write and a read to the allocated address.

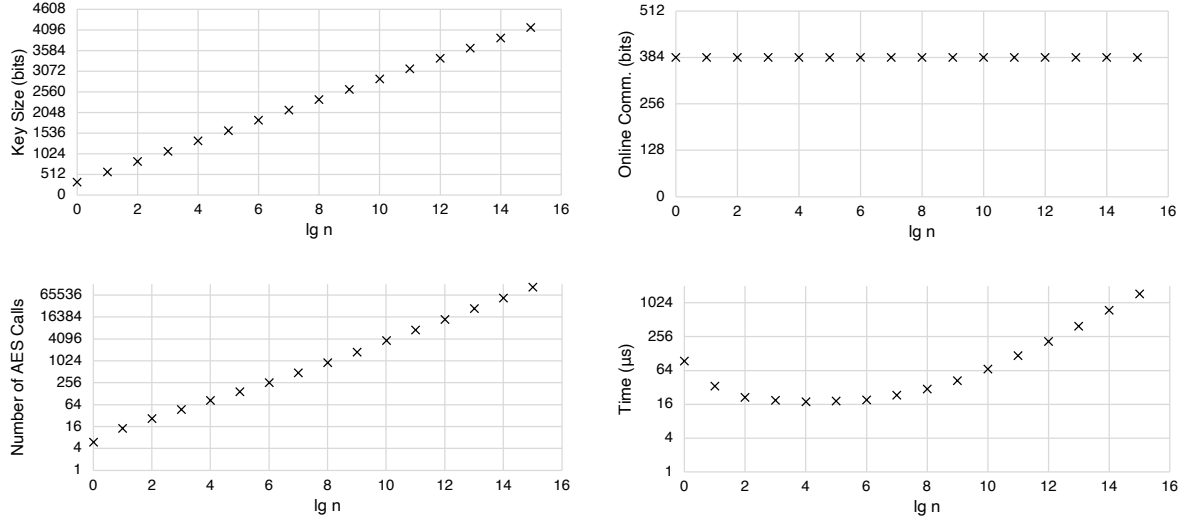


Figure 4: We initialized a SAM of size n , then executed n writes, followed by n reads. We consider a fixed word-size $w = 128$ bits. We plot amortized cost of a single read/write pair. In particular, we plot (top-left) the amount of preprocessing in the form of DPF keys and secret-shared permutations, (top-right) online communication cost, (bottom-left) number of calls to AES, and (bottom-right) online-phase runtime on a commodity machine.

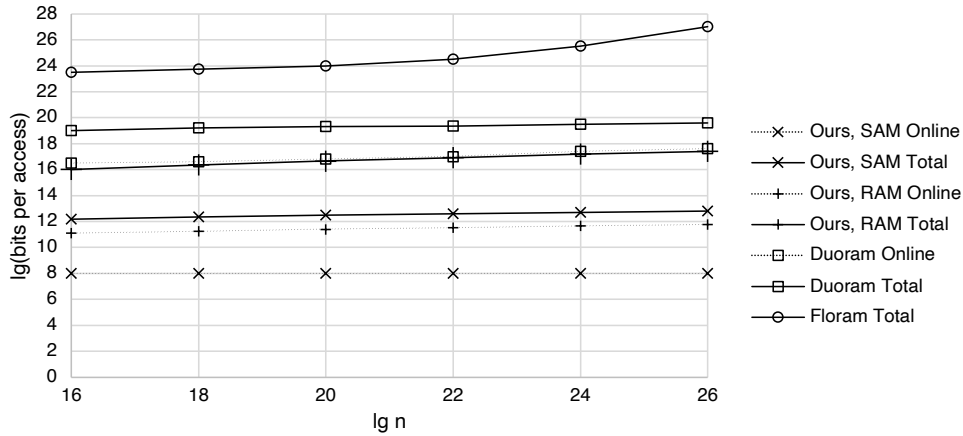


Figure 5: Comparison of bandwidth consumption of our approach as compared to that of Floram [Ds17] and 2PC Duoram [VHG23]. We plot the cost a single access of a 64-bit word. For our approach, we plot both the cost of a SAM access, and the cost of a RAM access, which we instantiate by a naïve binary tree of height $O(\lg n)$.

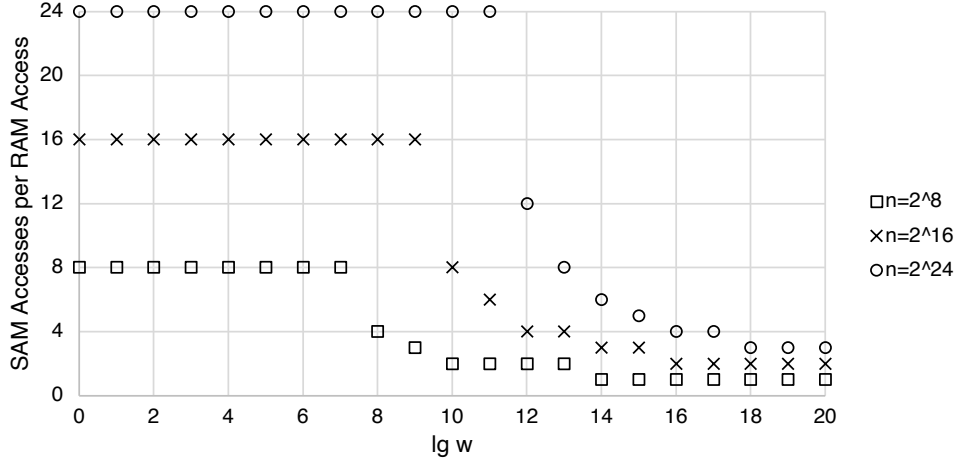


Figure 6: As discussed in Section 2.4, for large w , we can use SAM to efficiently implement RAM by means of a search trie with very high fan-out. We fix three choices of $n \in \{2^8, 2^{16}, 2^{24}\}$ and consider how many pairs of SAM read/write operations are needed to implement a single RAM access, as a function of w . Here, we set the degree of the RAM trie as $B = \max(2, w/\lg^2 n)$.

4.1 Protocol Costs

We implemented the online phase of our protocol in ≈ 500 lines of C++ (and ≈ 5800 lines of code needed for AES instruction compatibility issues). Our offline phase is instantiated as a semi-honest untrusted third party that generates secret-shared permutations and DPF keys. To achieve this, we instantiate the [BG116] DPF using AES as a PRG.

We ran our protocol by executing each of the two parties in a single-threaded environment. The two parties were run in parallel on a commodity laptop—a 2021 MacBook Pro with an Apple M1 Pro Chip and 16GB memory. We ran our approach on various SAM sizes n from 1 to 2^{15} , sufficient to establish a trend. We set the word size w to 128. Then, we ran a sequence of n writes followed by n reads. Figure 4 plots the amortized cost of each read/write pair in terms of (1) total amount of preprocessing (i.e., DPF keys and permutations), (2) online communication, (3) number of calls to AES, and (4) wall-clock runtime of the online phase.

Discussion Figure 4 confirms our analysis of the protocol. Communication costs of our protocol are lean, especially in the online phase. The plots also confirm the main bottleneck of our approach: the number of per-access calls to AES, which increases linearly in n . The runtime of our protocol is dominated by calls to AES, and (ignoring noise at very small n) there is clear linear trend in the per-access runtime.

4.2 Communication Cost Comparison

To compare to prior work, we examine the communication consumption of our protocol as compared to prior works Floram [Ds17], as well as Duoram’s 2PC approach [VHG23]; see Figure 5. The Floram and Duoram numbers are taken from [VHG23]. Here, we calculate our total communication cost when $w = 64$, matching the experiments of prior work. We also plot the communication cost needed to implement RAM, which we obtain in a straightforward way by implementing a binary tree; i.e., we ignore optimizations discussed in Section 2.4.

Discussion Figure 5 shows how lean our protocol is in terms of communication consumption. Our SAM accesses incur more than two orders of magnitude less online (resp. total) communication than that of Duoram. Of course, this is a slightly unfair comparison, since Duoram implements RAM, not SAM. Still,

when we use SAM to implement RAM using a binary tree, our communication cost remains lower than that of Duoram. In addition, our approach avoids the strong assumption of single-server PIR, which Duoram relies on.

4.3 Efficiency of RAM to SAM Compilation

Finally, following the discussion in Section 2.4, we consider implementing RAM with SAM, and examine the number pairs of SAM read/write operations needed to implement a single RAM operation as a function of word size, without asymptotically changing online bandwidth consumption.

Discussion Figure 6 reveals that as w increases, it is possible to implement RAM by SAM with a decreasing number of SAM accesses. This is mostly interesting when the word size is very large. We emphasize that in many cases, it is not necessary to compile RAM accesses to SAM, as a large variety of interesting tasks can be expressed directly as SAM programs [AHR24, AH25].

5 Related Work

Our emphasis below is on explaining and comparing to those works listed Table 1.

Oblivious RAM Traditionally, ORAM is defined in the client/server model, where the server acts as a simple read/write memory and performs no other computations [GO96]. In this setting, it is well known that in $\Theta(\lg n)$ communication blow-up is necessary [GO96, LN18] and sufficient [AKL⁺20].

In other settings, the $\Omega(\lg n)$ lower bound on communication blow-up *does not apply*. For instance, Onion ORAM [DvDF⁺16] achieved constant blow-up by leveraging *server-side computation* (namely, the server performs homomorphic encryption operations). The MPC setting also has no such bandwidth lower bound, which is how we are able to achieve constant factor blow-up.

RAM-MPC via Off-the-Shelf ORAM When using off-the-shelf ORAM to achieve an MPC protocol, client/server blow-up is not the relevant cost metric. Instead, one considers the *circuit complexity* of the client program. Under this metric, Circuit ORAM [WCS15] remains best, achieving $O((w + \lg^2 n)\lambda)$ amortized Boolean gates per access, where λ is a statistical security parameter controlling failure probability. [AHR24] showed that Circuit ORAM can be naturally simplified to achieve OSAM rather than ORAM, and the resulting client uses $O(w\lambda)$ amortized Boolean gates per access.

Client/Multi-Server Protocols As mentioned in Section 1, rather than using off-the-shelf ORAM, we can consider constructing custom protocols that use techniques unavailable in the basic client/server setting.

[OS97] pioneered this direction by considering a client/multi-server ORAM. The authors achieved feasibility results for different numbers of servers. For instance, with three servers, they achieved information-theoretic protocol with communication complexity $O(n^{1/3} \lg^3 n)$. While seminal in kickstarting the study of multi-server ORAM, this result was superseded by later single-server techniques, such as Path ORAM and Circuit ORAM.

[LO13a] later demonstrated an efficient two-server ORAM. By leveraging both servers, it is possible to achieve a relatively simple and efficient protocol. While the [LO13a] construction is good for their considered client/multi-server setting, it becomes prohibitively expensive when one emulates client behavior with an MPC protocol to achieve full RAM-MPC. Namely, their protocol requires that on each access the client invoke a cryptographic primitive $O(\lg n)$ times. In the 2PC setting, these become non-black-box invocations, which is expensive; see also Remark 1.

Custom RAM-MPC Protocols [ZWR⁺16] was, to our knowledge, the first work to investigate how to achieve full RAM-MPC with better performance than by using off-the-shelf ORAM. In particular, the authors modified the classic square-root complexity ORAM [GO96], replacing sorting networks by asymptotically-better *permutation networks*. The use of permutation networks does not make sense in the classic client/server

setting, because programming the network uses $O(n)$ space, but they *can* be used in the MPC setting, because the MPC parties can afford to run linear-space algorithms. This RAM-MPC achieved relatively poor $\tilde{O}(\sqrt{n})$ asymptotic complexity, but the authors showed that their customized protocol concretely outperforms asymptotically-better Circuit ORAM for a variety of parameters.

DPF-based Protocols As noted in Section 1, Floram [Ds17] was pivotal in RAM-MPC. The authors used DPFs to trade computation for improved communication. Their construction still consumes relatively high communication overhead, and it also involves some invocations of non-black-box cryptography, which we avoid.

Also as mentioned in Section 1, Duoram [VHG23] adopted Floram’s key ideas and used DPFs to achieve constant communication overhead for large word size in the 3PC. Our main result—constant-communication overhead—is thus similar to Duoram’s. However, to get their approach to work in the 2PC setting, Duoram increases communication overhead and resorts to using FHE-based private information retrieval, an expensive technique our approach avoids. Our approach achieves constant overhead in the 2PC setting, albeit for the weaker SAM model (or for the RAM model when working with very large words), and it relies only on OT.

Round-Efficient Protocols [JW18] studied RAM-MPC with a focus on *round complexity*. The authors observed that using Circuit ORAM to implement RAM-MPC incurs either (1) high round complexity, due to sequential computations inherent to the Circuit ORAM algorithm or (2) high communication complexity, due to the use of constant-round garbled-circuit-based techniques [Yao86]. The authors generated a 3PC instantiation of Circuit ORAM that leverages a number of custom improvements to achieve $O(\lg n)$ round complexity. For instance, the authors employ 3PC information-theoretic PIR to accelerate certain steps of lookup, and they use garbled-circuit-based techniques for the recursive position map. Their approach improves Circuit ORAM’s round complexity, but it inherits its relatively high communication complexity.

[BKKO20] constructed a 3PC DORAM with *constant* round complexity and sublinear communication. Their main contribution is a three-party DPF, which can be combined with two-party PIR protocols to achieve their stated asymptotic. The protocol requires linear computation.

Seeking to resolve the above tension between round complexity and computation cost, [HV21] proposed a 2PC-RAM protocol that achieves constant round complexity and *sublinear computation*. Their approach requires distributed evaluation of an oblivious pseudorandom function (OPRF, [FIPR05]). The authors also showed how to improve communication/computation further by employing a particular powerful primitive – doubly efficient private information retrieval (DEPIR, [BIM04]).

Garbled RAM [LO13b] solves the problem of round complexity in RAM-MPC by allowing the MPC circuit to *non-interactively* perform memory lookups. In the semi-honest model, the communication cost of all GRAM memory lookups can be moved to the offline phase. The most efficient GRAM [HKO23] is notable in that it achieves its cost by implementing the Circuit ORAM algorithm inside a Garbled Circuit [Yao86]. Because [AHR24] showed that Circuit ORAM can be simplified (by removing the position map) to construct “Circuit OSAM”, it is straightforward that [HKO23]’s GRAM can be simplified to construct “GSAM” at corresponding lower cost. Known approaches to GRAM/GSAM incur offline communication cost $\omega(w\lambda)$ where λ is a computational security parameter; notice that our communication *does not* include scaling in the product $w\lambda$.

Our approach has close to optimal round complexity – for each memory operation, parties exchange secret shares. Moreover, it is straightforward that our technique allows any number of non-dependent memory accesses to be performed in parallel, allowing further reduction in round consumption.

Other Techniques [FNO24] constructed a 3PC-RAM that achieves sub-logarithmic per-access communication overhead for large enough words w . The protocol is remarkable in that it does this without cryptographic assumptions. At the highest level, the authors achieve their cost by leveraging 3PC information-theoretic PIR.

References

- [AH25] Ananya Appan and David Heath. Oblivious immutable memory. Cryptology ePrint Archive, Report 2025/593, 2025.
- [AHR24] Ananya Appan, David Heath, and Ling Ren. Oblivious single access machines - A new model for oblivious computation. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 3080–3094. ACM Press, October 2024.
- [AKL⁺20] Gilad Asharov, Ilan Komargodski, Wei-Kai Lin, Kartik Nayak, Enoch Peserico, and Elaine Shi. OptORAMa: Optimal oblivious RAM. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 403–432. Springer, Cham, May 2020.
- [BCG⁺19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- [Ben64] Václav E Beneš. Permutation groups, complexes, and rearrangeable connecting networks. *Bell System Technical Journal*, 43(4):1619–1640, 1964.
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 337–367. Springer, Berlin, Heidelberg, April 2015.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303. ACM Press, October 2016.
- [BIM04] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers’ computation in private information retrieval: PIR with preprocessing. *Journal of Cryptology*, 17(2):125–151, March 2004.
- [BKKO20] Paul Bunn, Jonathan Katz, Eyal Kushilevitz, and Rafail Ostrovsky. Efficient 3-party distributed ORAM. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 215–232. Springer, Cham, September 2020.
- [Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.
- [Ds17] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 523–535. ACM Press, October / November 2017.
- [DvDF⁺16] Srinivas Devadas, Marten van Dijk, Christopher W. Fletcher, Ling Ren, Elaine Shi, and Daniel Wichs. Onion ORAM: A constant bandwidth blowup oblivious RAM. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part II*, volume 9563 of *LNCS*, pages 145–174. Springer, Berlin, Heidelberg, January 2016.
- [FIPR05] Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 303–324. Springer, Berlin, Heidelberg, February 2005.
- [FNO22] Brett Hemenway Falk, Daniel Noble, and Rafail Ostrovsky. 3-party distributed ORAM from oblivious set membership. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22*, volume 13409 of *LNCS*, pages 437–461. Springer, Cham, September 2022.
- [FNO24] Brett Hemenway Falk, Daniel Noble, and Rafail Ostrovsky. MetaDORAM: Info-theoretic distributed ORAM with less communication. Cryptology ePrint Archive, Report 2024/011, 2024.

- [GGM84] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th FOCS*, pages 464–479. IEEE Computer Society Press, October 1984.
- [GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 640–658. Springer, Berlin, Heidelberg, May 2014.
- [GKK⁺12] S. Dov Gordon, Jonathan Katz, Vladimir Kolesnikov, Fernando Krell, Tal Malkin, Mariana Raykova, and Yevgeniy Vahlis. Secure two-party computation in sublinear (amortized) time. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 513–524. ACM Press, October 2012.
- [GKW18] S. Dov Gordon, Jonathan Katz, and Xiao Wang. Simple and efficient two-server ORAM. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part III*, volume 11274 of *LNCS*, pages 141–157. Springer, Cham, December 2018.
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.
- [GO96] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.
- [HKO23] David Heath, Vladimir Kolesnikov, and Rafail Ostrovsky. Tri-state circuits - A circuit model that captures RAM. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 128–160. Springer, Cham, August 2023.
- [HV21] Ariel Hamlin and Mayank Varia. Two-server distributed ORAM with sublinear computation and constant rounds. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 499–527. Springer, Cham, May 2021.
- [IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 145–161. Springer, Berlin, Heidelberg, August 2003.
- [JW18] Stanislaw Jarecki and Boyang Wei. 3PC ORAM with low latency, low bandwidth, and fast batch retrieval. In Bart Preneel and Frederik Vercauteren, editors, *ACNS 2018*, volume 10892 of *LNCS*, pages 360–378. Springer, Cham, July 2018.
- [KM19] Eyal Kushilevitz and Tamer Mour. Sub-logarithmic distributed oblivious RAM with small block size. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part I*, volume 11442 of *LNCS*, pages 3–33. Springer, Cham, April 2019.
- [KU58] Andrei Nikolaevich Kolmogorov and Vladimir Andreevich Uspenskii. On the definition of an algorithm. *Uspekhi Matematicheskikh Nauk*, 13(4):3–28, 1958.
- [Lin16] Yehuda Lindell. How to simulate it - A tutorial on the simulation proof technique. Cryptology ePrint Archive, Report 2016/046, 2016.
- [LN18] Kasper Green Larsen and Jesper Buus Nielsen. Yes, there is an oblivious RAM lower bound! In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 523–542. Springer, Cham, August 2018.
- [LO13a] Steve Lu and Rafail Ostrovsky. Distributed oblivious RAM for secure two-party computation. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 377–396. Springer, Berlin, Heidelberg, March 2013.
- [LO13b] Steve Lu and Rafail Ostrovsky. How to garble RAM programs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 719–734. Springer, Berlin, Heidelberg, May 2013.

- [MR19] Daniel Masny and Peter Rindal. Endemic oblivious transfer. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 309–326. ACM Press, November 2019.
- [MW22] Samir Jordan Menon and David J. Wu. SPIRAL: Fast, high-rate single-server PIR via FHE composition. In *2022 IEEE Symposium on Security and Privacy*, pages 930–947. IEEE Computer Society Press, May 2022.
- [OS97] Rafail Ostrovsky and Victor Shoup. Private information storage (extended abstract). In *29th ACM STOC*, pages 294–303. ACM Press, May 1997.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022.
- [Sch80] Arnold Schönhage. Storage modification machines. *SIAM Journal on Computing*, 9(3):490–508, 1980.
- [SvDS⁺13] Emil Stefanov, Marten van Dijk, Elaine Shi, Christopher W. Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. Path ORAM: an extremely simple oblivious RAM protocol. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 299–310. ACM Press, November 2013.
- [VHG23] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duoram: A bandwidth-efficient distributed ORAM for 2- and 3-party computation. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 3907–3924. USENIX Association, August 2023.
- [Wak68] Abraham Waksman. A permutation network. *J. ACM*, 15(1):159–163, January 1968.
- [WCS15] Xiao Wang, T.-H. Hubert Chan, and Elaine Shi. Circuit ORAM: On tightness of the Goldreich-Ostrovsky lower bound. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 850–861. ACM Press, October 2015.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.
- [ZWR⁺16] Samee Zahur, Xiao Shaun Wang, Mariana Raykova, Adria Gascón, Jack Doerner, David Evans, and Jonathan Katz. Revisiting square-root ORAM: Efficient random access in multi-party computation. In *2016 IEEE Symposium on Security and Privacy*, pages 218–234. IEEE Computer Society Press, May 2016.